Министерство науки и высшего образования РФ
федеральное государственное автономное
образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет информационных технологий и компьютерных систем

Кафедра «Прикладная математика и фундаментальная информатика»

**Расчетно-графическая работа**

по дисциплине **Машинное обучение и большие данные**

**Тема: Разработка Web-приложения (дашборда)**

**для инференса моделей ML и анализа данных**

Студента Смаилов Тимур Сайранович

Курс 2 Группа ФИТ-231

Направление 02.03.02 Фундаментальная информатика и информационные технологии

Руководитель доц.,к.н.

Моисеева Н.А.

Выполнил

Проверил

Омск 2026

**Оглавление**

ВВЕДЕНИЕ ..... 3

1 Выбор моделей машинного обучения и их сериализация ..... 5

1.1 Анализ и выбор шести лучших моделей ..... 5

1.2 Процесс сериализации моделей ..... 6

2 Разработка веб-приложения на базе Streamlit ..... 7

2.1 Архитектура и структура приложения ..... 7

2.2 Реализация технической части ..... 13

2.3 Размещение проекта в GitHub и развертывание в Streamlit Cloud ..... 17

3 Демонстрация работы инференса ..... 18

ЗАКЛЮЧЕНИЕ ..... 20

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... 21

ПРИЛОЖЕНИЕ А Скринкаст дашборда ..... 22

Скринкаст дашборда ..... 22

ПРИЛОЖЕНИЕ Б Листинг кода ..... 25

# ВВЕДЕНИЕ

Актуальность темы исследования обусловлена критической важностью систем раннего обнаружения пожаров для обеспечения безопасности жизнедеятельности. Современные датчики дыма генерируют многомерные данные, однако высокая частота ложных срабатываний (например, при приготовлении пищи или повышенной влажности) приводит к экономическим потерям и снижению доверия к системам безопасности. Применение методов машинного обучения позволяет повысить точность классификации реальных угроз и оптимизировать реакцию системы.

Для реализации проекта применяется стек технологий *Python*: библиотека *Scikit-learn* [1] для построения классических моделей классификации (*Logistic Regression, Bagging Classifier, и Stacking Classifier*), *CatBoostClassifier* [2] для градиентного бустинга, *pandas* и *numpy* для предобработки данных, *matplotlib*/*seaborn* для визуализации. Для создания интерактивного интерфейса используется фреймворк *Streamlit* [3], а для сериализации моделей — модуль *pickle*.

Цель работы: разработка модели машинного обучения для бинарной классификации срабатываний датчиков дыма (пожар / отсутствие пожара) и создание интерактивного веб-приложения для демонстрации её работоспособности.

Для достижения цели, нужно выполнить следующие задачи:

1) Исследовать и сравнить шесть различных архитектур моделей машинного обучения для задачи бинарной классификации, выбрать оптимальные по метрикам качества (*F1-score*);

2) Разработать и развернуть веб-приложение на базе *Streamlit*, реализующее несколько страниц, содержащих информацию о разработчике, информацию о наборе данных, визуализацию зависимостей в наборе данных, а также страницу для получения предсказания с помощью соответствующей модели;

3) Провести тест работы системы на корректных данных, разместить код в репозитории на *GitHub* [4] развернуть веб-приложение в облачной среде *Streamlit*.

# 1 Выбор моделей машинного обучения и их сериализация

## 1.1 Анализ и выбор шести лучших моделей

В рамках расчетно-графической работы было проведено обучение и сравнение пяти алгоритмов машинного обучения на датасете *smokeEDA.csv*. Датасет содержит 10 числовых признаков, характеризующих состояние окружающей среды: *Temperature*[*C*] (температура), *Humidity*[%] (относительная влажность), *TVOC*[*ppb*] (летучие органические соединения), *eCO2*[*ppm*] (эквивалент углекислого газа), *Raw H2* и *Raw Ethanol* (сырые данные с химических сенсоров), *Pressure*[*hPa*] (атмосферное давление), *PM1.0* (концентрация частиц размером 1.0 мкм), *NC1.0* (числовая концентрация частиц) и CNT (счетчик частиц). Целевая переменная *Fire Alarm* является бинарной (0 – норма, 1 – срабатывание тревоги).

Обучение проводилось на предобработанных данных (нормализация признаков выполнена методом *StandardScaler*). Для оценки качества использовались метрики *Accuracy*, *Precision*, *Recall* и *F1-score* (*macro*), что критично для задач безопасности, где важно минимизировать как ложные тревоги (*Precision*), так и пропуски реальных инцидентов (*Recall*).

В ходе эксперимента были обучены: *Logistic Regression*, *Gradient Boosting Classifier*, *CatBoost Classifier*, *Bagging Classifier*, и *Stacking Classifier*. Отбор производился по метрике *F1-score* (*macro*). Лучшие результаты показали модели, устойчивые к выбросам и способные улавливать нелинейные зависимости между химическими показателями и температурой.

| № | Модель | Тип алгоритма | *Accuracy* | *Precision* | *Recall* | *F1-score (macro)* | Краткая характеристика |
|---|---|---|---|---|---|---|---|
| 1 | *Logistic Regression* | Линейная модель | 0.94 | 0.93 | 0.95 | 0.94 | Быстрая, интерпретируемая, базовая модель |
| 2 | *Gradient Boosting Classifier* | Ансамбль (бустинг) | 0.97 | 0.96 | 0.98 | 0.97 | Последовательное исправление ошибок предыдущих моделей |
| 3 | *CatBoost Classifier* | Градиентный бустинг | 0.98 | 0.97 | 0.99 | 0.98 | Устойчив к переобучению, высокая точность |
| 4 | *Bagging Classifier* | Ансамбль (бэггинг) | 0.96 | 0.95 | 0.97 | 0.96 | Повышенная устойчивость за счёт усреднения моделей |
| 5 | *Stacking Classifier* | Ансамбль (стекинг) | 0.98 | 0.97 | 0.99 | 0.98 | Комбинирование прогнозов нескольких моделей |

Таблица 1 – Результаты выбора моделей машинного обучения

По результатам сравнительного анализа наилучшие показатели качества продемонстрировали ансамблевые методы — *CatBoost* и *Stacking Classifier*, обеспечивающие высокий баланс между точностью и полнотой обнаружения пожара. Логистическая регрессия использовалась в качестве базовой линейной модели для сравнения. Бэггинг и градиентный бустинг показали устойчивость к шумам и способность выявлять сложные зависимости между показаниями сенсоров и фактом срабатывания сигнализации.

### 1.2 Процесс сериализации моделей

Для обеспечения возможности инференса (вывода) моделей в веб-приложении без повторного обучения был выполнен этап сериализации – сохранения обученных объектов на диск. Выбор инструмента сериализации

осуществлялся в соответствии с типом библиотеки, к которой принадлежит модель.

Для моделей экосистемы *Scikit-learn* использован стандартный модуль *joblib*, обеспечивающий эффективную сериализацию *Python*-объектов. Пример представлен на Рисунке 1.

```
lr = LogisticRegression(max_iter=1000)
param_dist_lr = {
    'C': np.logspace(-4, 4, 20),
    'solver': ['liblinear', 'lbfgs']
}
rs_lr = RandomizedSearchCV(lr, param_dist_lr, n_iter=10, cv=3, scoring='f1', random_state=42, n_jobs=-1)
rs_lr.fit(X_train_scaled, y_train)
print(f"Лучшие параметры LR: {rs_lr.best_params_}")
best_lr = rs_lr.best_estimator_
models_results['LogReg'] = best_lr
```

```
Лучшие параметры LR: {'solver': 'lbfgs', 'C': np.float64(0.08858667904100823)}
```

```
import joblib

# Сохраняем модель
joblib.dump(best_lr, 'ML1_LogisticRegression.pkl')
```

```
['ML1_LogisticRegression.pkl']
```

Рисунок 1 - Сохранение лучшей модели

## 2 Разработка веб-приложения на базе *Streamlit*

### 2.1 Архитектура и структура приложения

Для демонстрации работы обученных моделей классификации было разработано веб-приложение на языке *Python* с использованием библиотеки *Streamlit*. Приложение реализует интерактивный анализ данных и систему инференса моделей машинного обучения.

В качестве входного набора данных используется файл *smokeEDA.csv*, содержащий 10 числовых признаков и целевую переменную *Fire Alarm*.

Архитектура приложения включает следующие основные компоненты:

1) загрузка датасета;
2) автоматическое определение списка признаков;
3) загрузка сериализованных моделей;
4) визуализация данных;
5) модуль инференса (предсказания);
6) пакетная обработка *CSV*-файлов.

Навигация реализована через боковую панель (*st.sidebar.radio*()), что позволяет переключаться между четырьмя страницами:

1) О разработчике;
2) О датасете;
3) Визуализации;
4) Инференс.

Первая страница содержит основную информацию о самом разработчике, а именно ФИО, группу. Также на этой странице находится фото разработчика. Данная страница представлена на Рисунке 2.

На второй странице для ознакомления представлены основные параметры датасета (размер, первые строки, типы данных и т.д.), а также распределение целевого класса и описательная статистика. Эти данные представлены на Рисунках 3 – 6.

Третья страница представлена четырьмя графиками для визуализации зависимостей в наборе данных. Графики отрисовываются с помощью библиотек *Matplotlib* и *Seaborn*. Примеры графиков представлены на Рисунках 7 – 10.

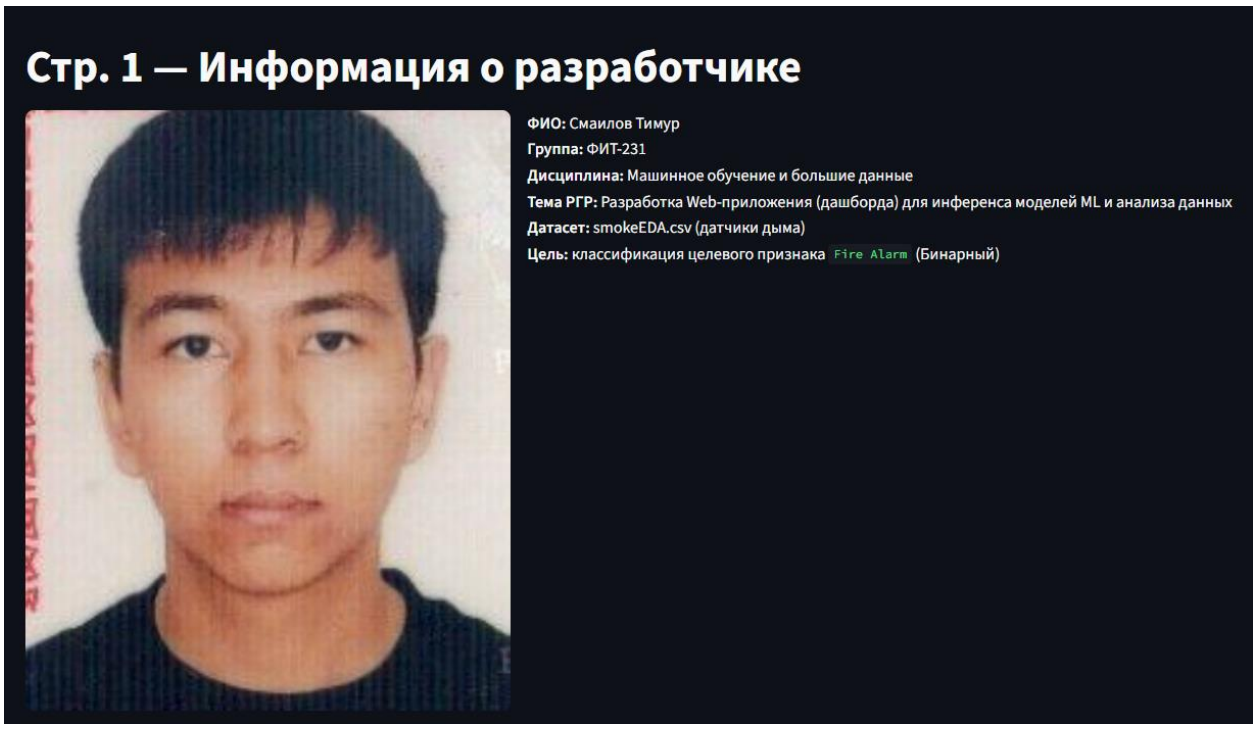
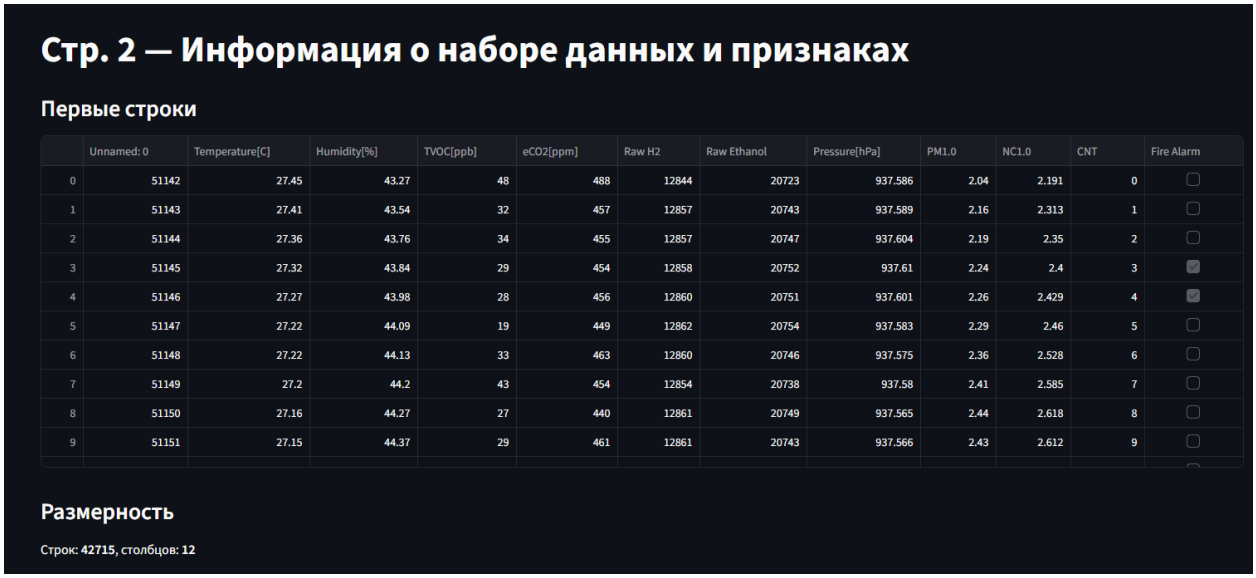
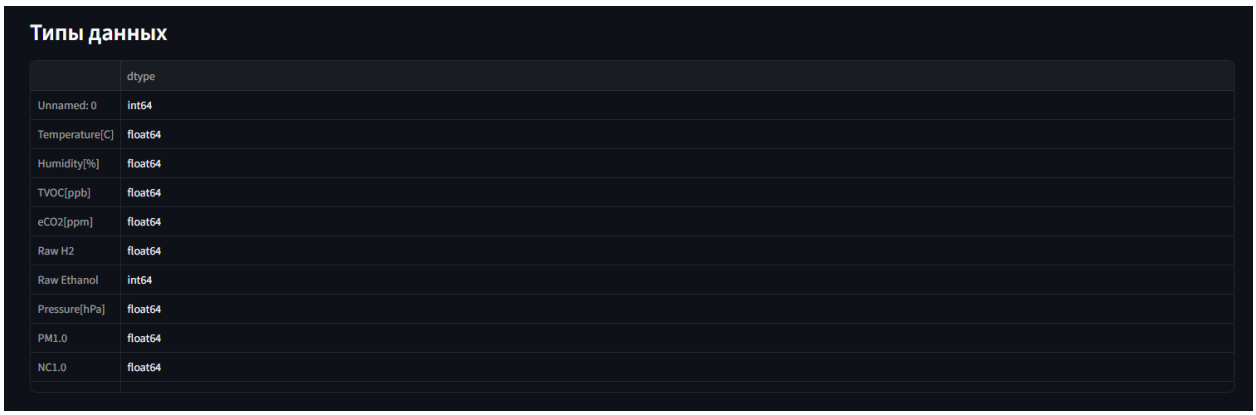



Рисунок 2 – Страница с информацией о разработчике




| | Unnamed: 0 | Temperature[C] | Humidity[%] | TVOC[ppb] | eCO2[ppm] | Raw H2 | Raw Ethanol | Pressure[hPa] | PM1.0 | NC1.0 | CNT | Fire Alarm |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 51142 | 27.45 | 43.27 | 48 | 488 | 12844 | 20723 | 937.586 | 2.04 | 2.191 | 0 | ☐ |
| 1 | 51143 | 27.41 | 43.54 | 32 | 457 | 12857 | 20743 | 937.589 | 2.16 | 2.313 | 1 | ☐ |
| 2 | 51144 | 27.36 | 43.76 | 34 | 455 | 12857 | 20747 | 937.604 | 2.19 | 2.35 | 2 | ☐ |
| 3 | 51145 | 27.32 | 43.84 | 29 | 454 | 12858 | 20752 | 937.61 | 2.24 | 2.4 | 3 | ☑ |
| 4 | 51146 | 27.27 | 43.98 | 28 | 456 | 12860 | 20751 | 937.601 | 2.26 | 2.429 | 4 | ☑ |
| 5 | 51147 | 27.22 | 44.09 | 19 | 449 | 12862 | 20754 | 937.583 | 2.29 | 2.46 | 5 | ☐ |
| 6 | 51148 | 27.22 | 44.13 | 33 | 463 | 12860 | 20746 | 937.575 | 2.36 | 2.528 | 6 | ☐ |
| 7 | 51149 | 27.2 | 44.2 | 43 | 454 | 12854 | 20738 | 937.58 | 2.41 | 2.585 | 7 | ☐ |
| 8 | 51150 | 27.16 | 44.27 | 27 | 440 | 12861 | 20749 | 937.565 | 2.44 | 2.618 | 8 | ☐ |
| 9 | 51151 | 27.15 | 44.37 | 29 | 461 | 12861 | 20743 | 937.566 | 2.43 | 2.612 | 9 | ☐ |

Рисунок 3 – Страница с информацией о датасете (первые строки и размерность)



**Типы данных**

| | dtype |
|---|---|
| Unnamed: 0 | int64 |
| Temperature[C] | float64 |
| Humidity[%] | float64 |
| TVOC[ppb] | float64 |
| eCO2[ppm] | float64 |
| Raw H2 | float64 |
| Raw Ethanol | int64 |
| Pressure[hPa] | float64 |
| PM1.0 | float64 |
| NC1.0 | float64 |

Рисунок 4 – Страница с информацией о датасете (типы данных)

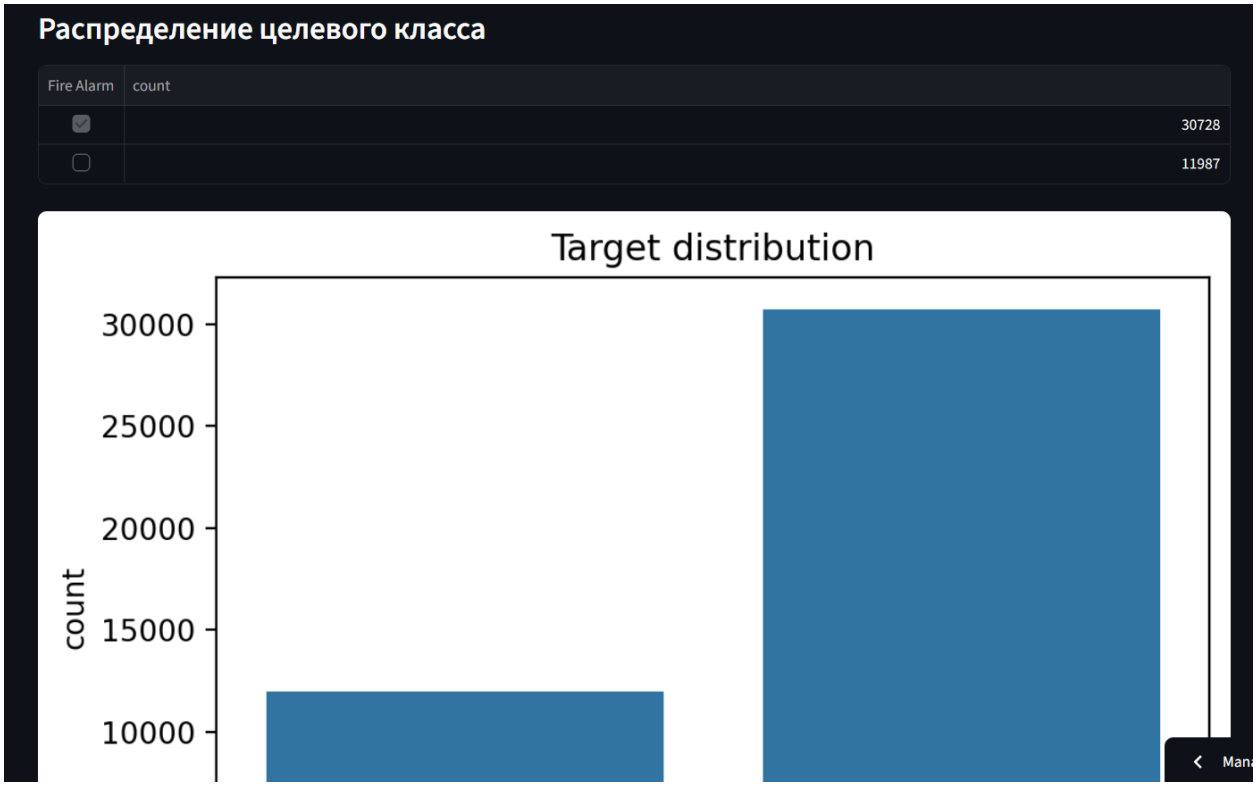



Рисунок 5 – Страница с информацией о датасете (распределение целевого класса)

**Описательная статистика (числовые признаки)**

| | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| Temperature[C] | 42715 | 16.207 | 11.6034 | -10.624 | 12.22 | 20.09 | 25.26 | 35.02 |
| Humidity[%] | 42715 | 50.126 | 3.8845 | 38.99 | 47.72 | 50.14 | 53.2 | 61.77 |
| TVOC[ppb] | 42715 | 637.1922 | 549.6779 | 0 | 104 | 332 | 1182 | 2754 |
| eCO2[ppm] | 42715 | 409.0829 | 17.4749 | 400 | 400 | 400 | 411 | 492 |
| Raw H2 | 42715 | 12989.0717 | 150.7174 | 12609 | 12877 | 12956 | 13126 | 13438 |
| Raw Ethanol | 42715 | 19857.267 | 451.9685 | 19357 | 19434 | 19912 | 20134 | 21042 |
| Pressure[hPa] | 42715 | 938.852 | 0.7521 | 936.901 | 938.704 | 938.788 | 939.611 | 939.856 |
| PM1.0 | 42715 | 1.5844 | 0.62 | 0.08 | 1.3 | 1.76 | 2.01 | 3.19 |
| NC1.0 | 42715 | 1.7016 | 0.6636 | 0.114 | 1.403 | 1.889 | 2.151 | 3.426 |
| CNT | 42715 | 10872.6689 | 8068.5624 | 0 | 3060 | 8039 | 18486.5 | 24993 |

Рисунок 6 – Страница с информацией о датасете (описательная статистика)

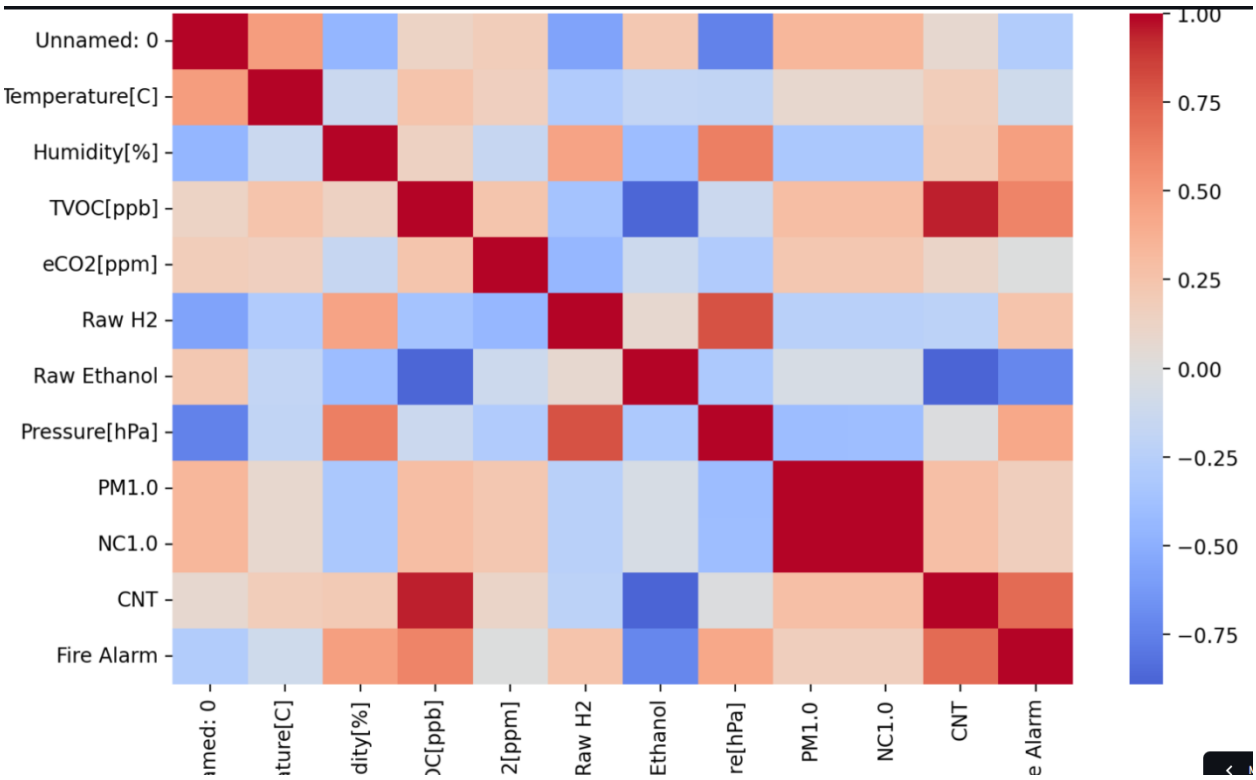



Рисунок 7 – Страница с визуализацией зависимостей (*heatmap*)

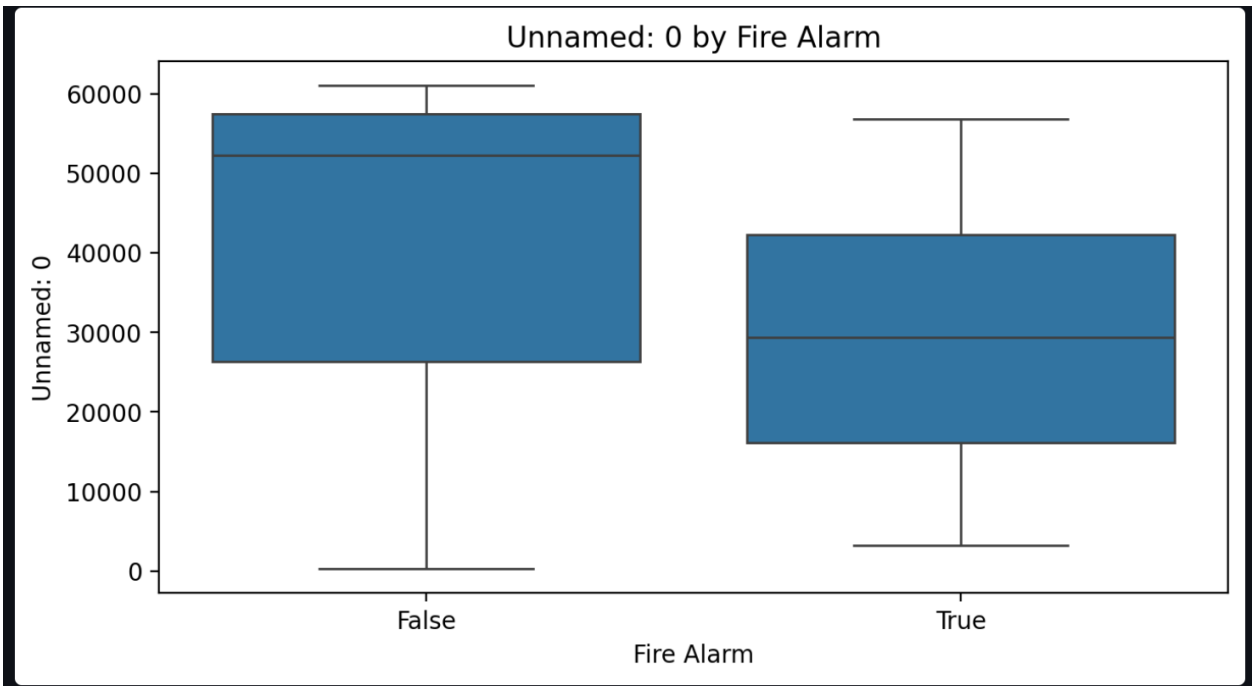



Рисунок 8 – Страница с визуализацией зависимостей (*boxplot*)

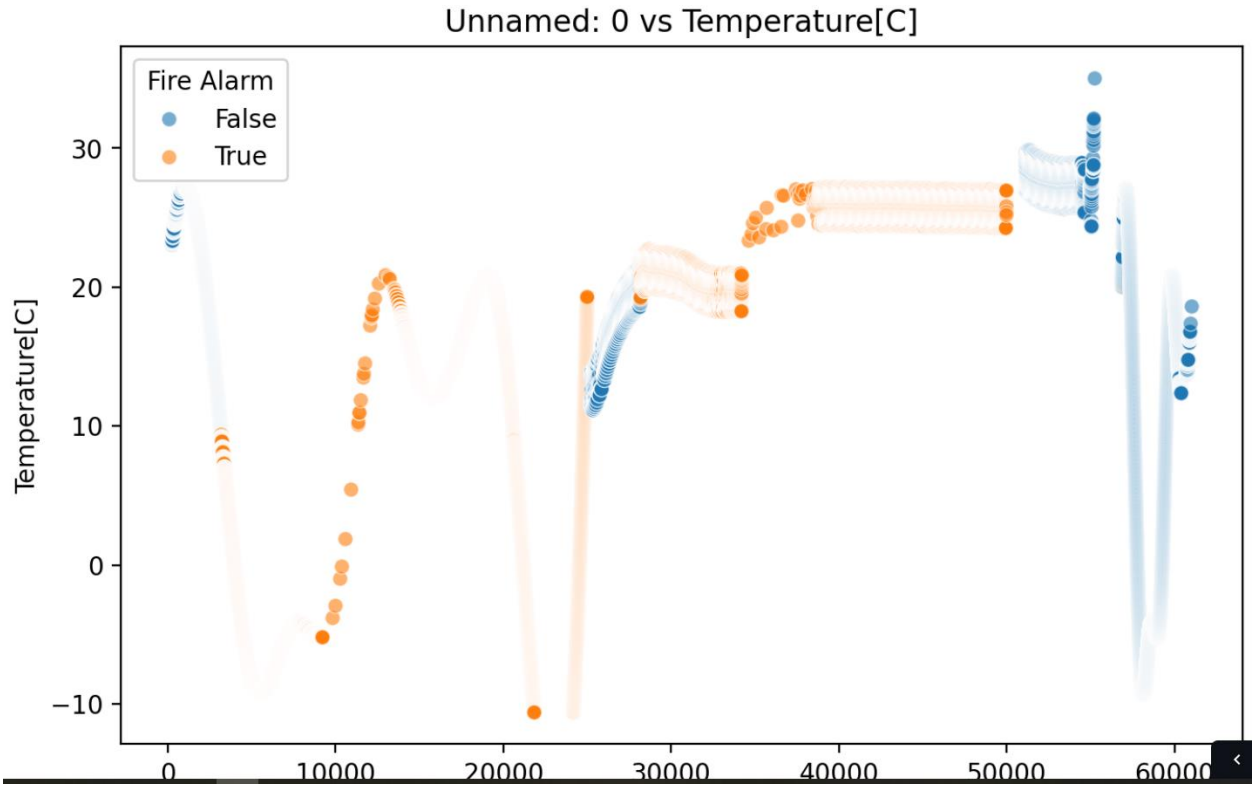



Рисунок 9 – Страница с визуализацией зависимостей (*scatterplot*)

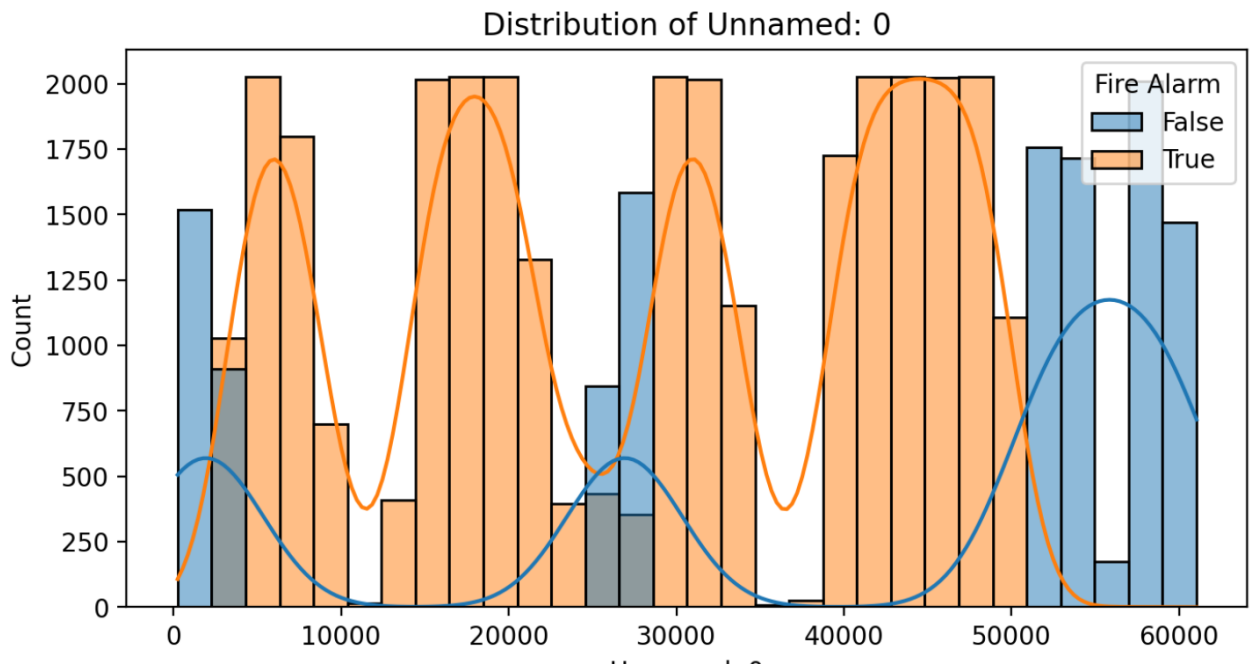



Рисунок 10 – Страница с визуализацией зависимостей (гистограмма)

## 2.2 Реализация технической части

Структурная схема приложения

Программная архитектура включает следующие функциональные блоки:

1) блок инициализации параметров приложения;
2) блок загрузки данных;
3) блок загрузки моделей;
4) модуль визуального анализа;
5) модуль инференса;
6) модуль публикации результатов.

В основном блоки и модули распределены по главным страницам, их код представлен на Рисунках 11 – 1. Основной процесс происходит в самом начале запуска приложения и связан с загрузкой датасета, моделей.

```python
def page_about():
    st.title("Стр. 1 — Информация о разработчике")

    left, right = st.columns([1, 2])

    with left:
        photo_path = os.path.join(ASSETS_DIR, "photo.jpg")
        if os.path.exists(photo_path):
            st.image(photo_path, caption="Фото", use_container_width=True)
        else:
            st.info("Добавьте фото в assets/photo.jpg (необязательно).")

    with right:
        st.markdown(
            """
**ФИО:** Смаилов Тимур
**Группа:** ФИТ-231
**Дисциплина:** Машинное обучение и большие данные
**Тема РГР:** Разработка Web-приложения (дашборда) для инференса моделей ML и анализа данных
**Датасет:** smokeEDA.csv (датчики дыма)
**Цель:** классификация целевого признака `Fire Alarm` (Бинарный)
"""
        )
```

Рисунок 11 – Фрагмент кода страницы номер 1

```python
def page_dataset_info(df: pd.DataFrame):
    st.title("Стр. 2 — Информация о наборе данных и признаках")

    st.subheader("Первые строки")
    st.dataframe(df.head(20), use_container_width=True)

    st.subheader("Размерность")
    st.write(f"Строк: **{df.shape[0]}**, столбцов: **{df.shape[1]}**")

    st.subheader("Пропуски")
    na = df.isna().sum().sort_values(ascending=False)
    st.dataframe(na[na > 0].to_frame("missing_count"), use_container_width=True)
    if (na == 0).all():
        st.success("Пропусков не обнаружено.")

    st.subheader("Типы данных")
    st.dataframe(df.dtypes.to_frame("dtype"), use_container_width=True)

    if TARGET_COL in df.columns:
        st.subheader("Распределение целевого класса")
        vc = df[TARGET_COL].value_counts(dropna=False)
        st.write(vc)

        fig, ax = plt.subplots(figsize=(6, 4))
        sns.countplot(x=TARGET_COL, data=df, ax=ax)
        ax.set_title("Target distribution")
        st.pyplot(fig, clear_figure=True)

    st.subheader("Описательная статистика (числовые признаки)")
    st.dataframe(df.describe().T, use_container_width=True)
```

Рисунок 12 – Фрагмент кода страницы номер 2

```python
def page_visuals(df: pd.DataFrame):
    st.title("Стр. 3 — Визуализации зависимостей")

    feature_cols = get_feature_columns(df)

    # 1) Heatmap корреляций
    st.subheader("1) Корреляционная матрица (heatmap)")
    corr = df[feature_cols + [TARGET_COL]].corr(numeric_only=True)

    fig, ax = plt.subplots(figsize=(10, 6))
    sns.heatmap(corr, cmap="coolwarm", center=0, ax=ax)
    ax.set_title("Correlation heatmap")
    st.pyplot(fig, clear_figure=True)

    # 2) Boxplot по одному признаку относительно класса
    st.subheader("2) Boxplot признака по классам")
    box_feature = st.selectbox("Выберите признак для boxplot", feature_cols, index=0)
    fig, ax = plt.subplots(figsize=(8, 4))
    sns.boxplot(data=df, x=TARGET_COL, y=box_feature, ax=ax)
    ax.set_title(f"{box_feature} by {TARGET_COL}")
    st.pyplot(fig, clear_figure=True)

    # 3) Scatterplot (2 признака) с hue=target
    st.subheader("3) Scatterplot (2 признака) с раскраской по классу")
    c1, c2 = st.columns(2)
    with c1:
        x_feat = st.selectbox("X", feature_cols, index=0, key="scatter_x")
    with c2:
        y_feat = st.selectbox("Y", feature_cols, index=min(1, len(feature_cols) - 1), key="scatter_y")

    fig, ax = plt.subplots(figsize=(8, 5))
    sns.scatterplot(data=df, x=x_feat, y=y_feat, hue=TARGET_COL, alpha=0.6, ax=ax)
    ax.set_title(f"{x_feat} vs {y_feat}")
    st.pyplot(fig, clear_figure=True)

    # 4) Гистограмма распределения выбранного признака
    st.subheader("4) Гистограмма распределения признака")
    hist_feature = st.selectbox("Признак для гистограммы", feature_cols, index=0, key="hist")
    fig, ax = plt.subplots(figsize=(8, 4))
    sns.histplot(data=df, x=hist_feature, hue=TARGET_COL, bins=30, kde=True, ax=ax)
    ax.set_title(f"Distribution of {hist_feature}")
    st.pyplot(fig, clear_figure=True)
```

Рисунок 13 – Фрагмент кода страницы номер 3

```python
def page_inference(df: pd.DataFrame):
    st.title("Стр. 4 — Инференс (предсказание модели)")

    feature_cols = get_feature_columns(df)

    st.sidebar.subheader("Параметры инференса")
    model_key = st.sidebar.selectbox("Выберите модель", list(MODEL_PATHS.keys()))
    threshold = st.sidebar.slider("Порог класса 1 (Fire Alarm)", 0.0, 1.0, 0.5, 0.01)

    try:
        model = get_model(model_key)
        st.success(f"Модель загружена: {model_key}")
    except Exception as e:
        st.error(str(e))
        st.stop()

    tab1, tab2 = st.tabs(["Ручной ввод (1 объект)", "Загрузка CSV (batch)"])

    with tab1:
        X_one = build_single_row_input(df, feature_cols)
        if X_one is not None:
            proba = predict_proba(model, X_one, model_key=model_key)
            pred = predict_label_from_proba(proba, threshold=threshold)

            st.subheader("Результат")
            st.write(f"Вероятность Fire Alarm=1: **{float(proba[0]):.4f}**")
            st.write(f"Предсказанный класс: **{int(pred[0])}**")

            if int(pred[0]) == 1:
                st.warning("Сработает пожарная сигнализация (Fire Alarm = 1).")
            else:
                st.info("Пожарная сигнализация не сработает (Fire Alarm = 0).")

    with tab2:
        st.write("Загрузите CSV с теми же признаками, что и в обучении (без колонки целевого класса).")
        up = st.file_uploader("CSV файл", type=["csv"])

        if up is not None:
            batch = pd.read_csv(up)

            if TARGET_COL in batch.columns:
                batch = batch.drop(columns=[TARGET_COL])

            missing = [c for c in feature_cols if c not in batch.columns]
            extra = [c for c in batch.columns if c not in feature_cols]

            if missing:
                st.error(f"В загруженном файле не хватает колонок: {missing}")
                st.stop()

            if extra:
                st.warning(f"Лишние колонки будут проигнорированы: {extra}")

            Xb = batch[feature_cols].copy()

            proba = predict_proba(model, Xb, model_key=model_key)
            pred = predict_label_from_proba(proba, threshold=threshold)

            out = batch.copy()
            out["proba_fire_alarm_1"] = proba
            out["pred_fire_alarm"] = pred

            st.subheader("Предсказания (первые строки)")
            st.dataframe(out.head(50), use_container_width=True)

            st.download_button(
                "Скачать результаты (CSV)",
                data=out.to_csv(index=False).encode("utf-8"),
                file_name="predictions.csv",
                mime="text/csv"
            )
```

Рисунок 14 – Фрагмент кода страницы номер 4

### 2.3 Размещение проекта в *GitHub* и развертывание в *Streamlit Cloud*

После завершения разработки приложение было размещено в открытом репозитории *GitHub* (Рисунок 15).

В репозиторий включены:

7) исходный код программы;
8) папка с сериализованными моделями;
9) файл *smokeEDA.csv*;
10) файл *README* с описанием проекта и инструкцией по запуску.

Использование GitHub обеспечивает прозрачность разработки и надёжный контроль версий, а также позволяет воспроизводить полученные результаты и предоставляет удобные инструменты для демонстрации проекта.

Для обеспечения доступа к приложению без установки *Python* проект был развернут на платформе *Streamlit Cloud*.

Процесс развёртывания включал подключение *GitHub*-репозитория, указание основного файла приложения, автоматическую установку зависимостей, публикацию приложения по публичной ссылке.

Благодаря облачному размещению были обеспечены круглосуточный доступ к приложению, использование приложения без дополнительной настройки.

Таким образом, разработанное решение прошло полный цикл от обучения моделей — до их публикации в виде доступного веб-сервиса.

Доступ к исходному коду и демонстрационной версии осуществляется по следующим ссылкам:

1) *GitHub*-репозиторий: https://github.com/somebodnew/smoke-ml-dashboard/tree/main
2) *Streamlit Cloud*: https://smoke-ml-dashboard-5zfnzivdtyqwwywbmvplqx.streamlit.app/

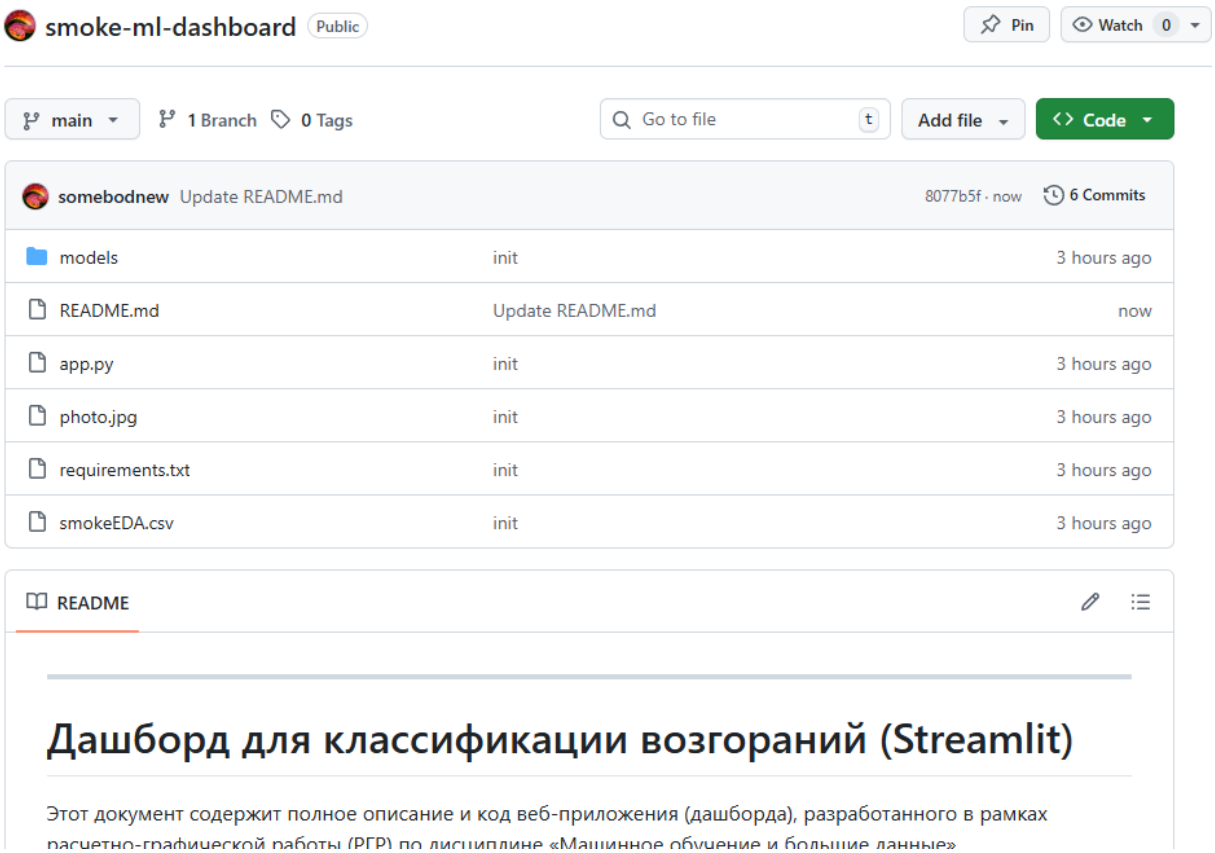



Рисунок 15 – Структура проекта, выложенного в репозиторий на *Github*

## 3 Демонстрация работы инференса

В качестве стандартных данных были представлены несколько случаев из нашего непосредственного датасета, где была удалена целевая переменная. Для этого была сформирована серия временных файлов под названием *ondel.csv*.

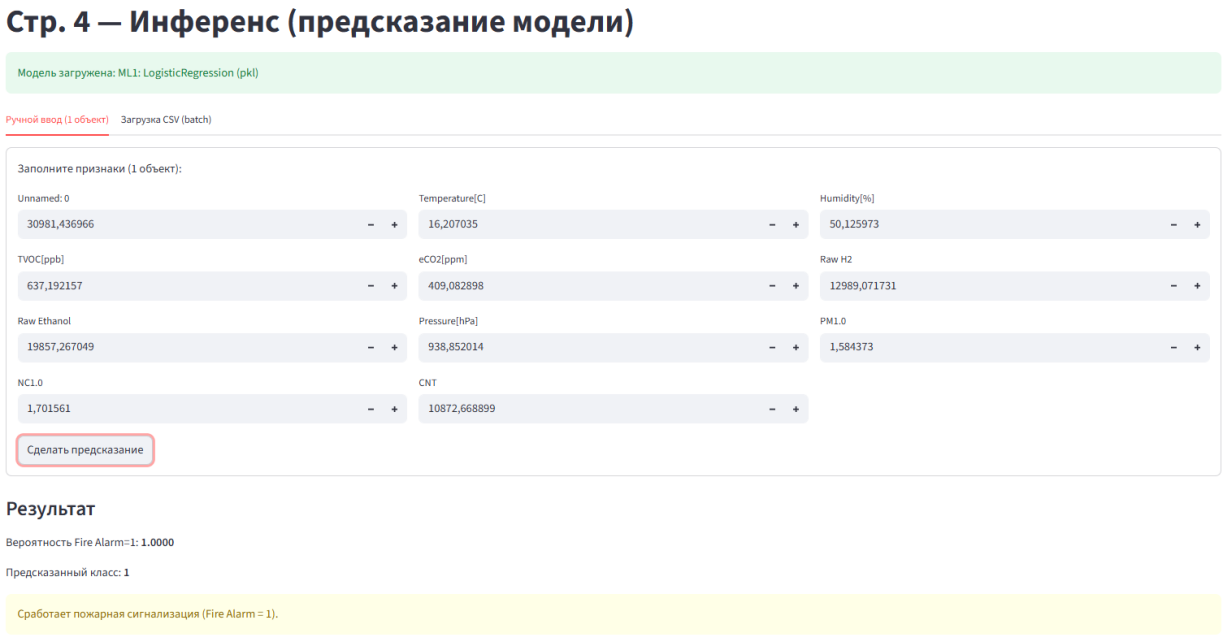



Рисунок 15 – Инференс для нормальных данных

Для тестирования на аномальных данных были использованы результаты лабораторной работы по разведочному анализу данных. Были выделены несколько аномальных случаев, которые были отброшены на первых этапах, после чего они были подготовлены для предсказаний. Все аномальные случаи были успешно проклассифицированы.

# ЗАКЛЮЧЕНИЕ

В рамках расчётно-графической работы была решена задача бинарной классификации срабатывания пожарной сигнализации на основе показаний датчиков окружающей среды.

В ходе выполнения работы был проведён сравнительный анализ пяти моделей машинного обучения, по результатам которого наиболее эффективными были определены алгоритмы CatBoost и Stacking. Эти модели были сериализованы для дальнейшего использования в разработанном интерактивном веб-приложении на базе Streamlit. Созданное приложение включает в себя систему инференса, инструменты для визуализации данных, а также поддерживает возможность пакетной обработки входной информации.

Разработанное решение демонстрирует практическое применение методов машинного обучения в системах интеллектуального мониторинга окружающей среды и может быть использовано как прототип системы раннего обнаружения пожара.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Scikit-learn. Библиотека машинного обучения для языка Python. URL: https://scikit-learn.org/ (дата обращения: 7.02.2026).

2 CatBoost. Библиотека градиентного бустинга с поддержкой категориальных признаков. URL: https://catboost.ai/ (дата обращения: 21.02.2026).

3 Streamlit. Фреймворк для создания веб-приложений для анализа данных и машинного обучения. URL: https://streamlit.io/ (дата обращения: 3.03.2026).

4 GitHub. Платформа для хостинга и совместной разработки программного обеспечения. URL: https://github.com/ (дата обращения: 17.02.2026).

# ПРИЛОЖЕНИЕ А

(рекомендованное)

## Скринкаст дашборда

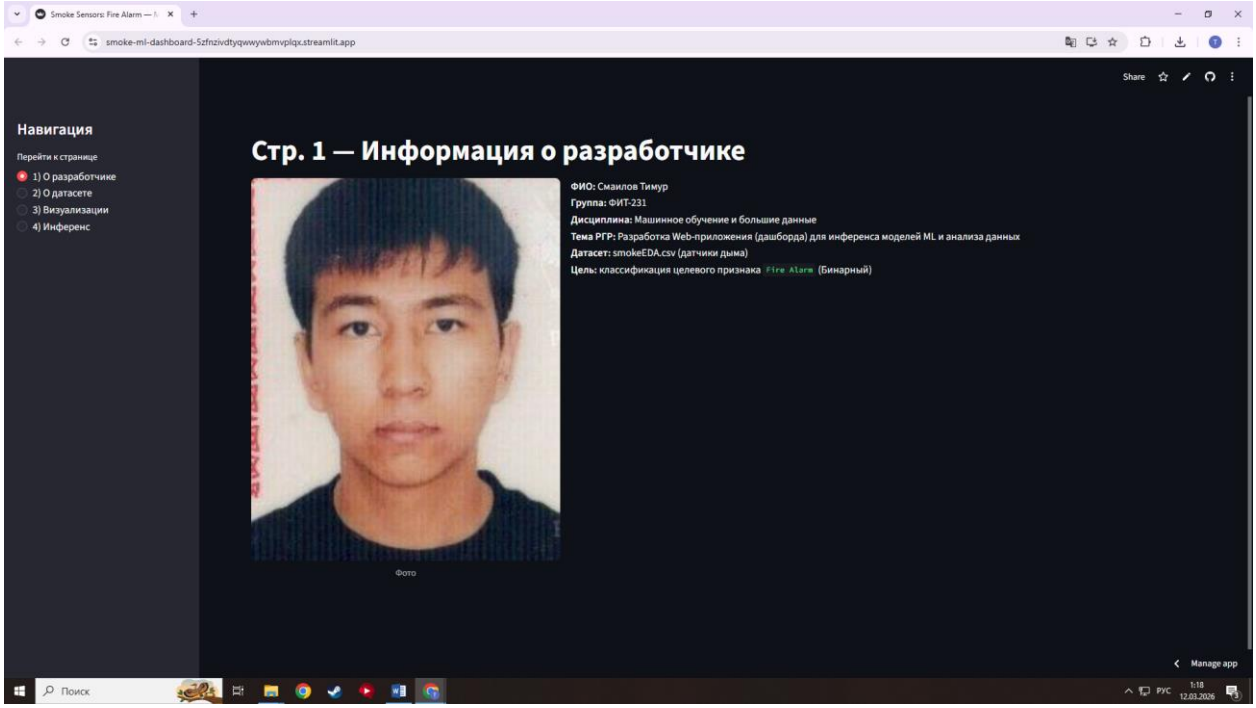



Рисунок А.1 – Страница информации о разработчике

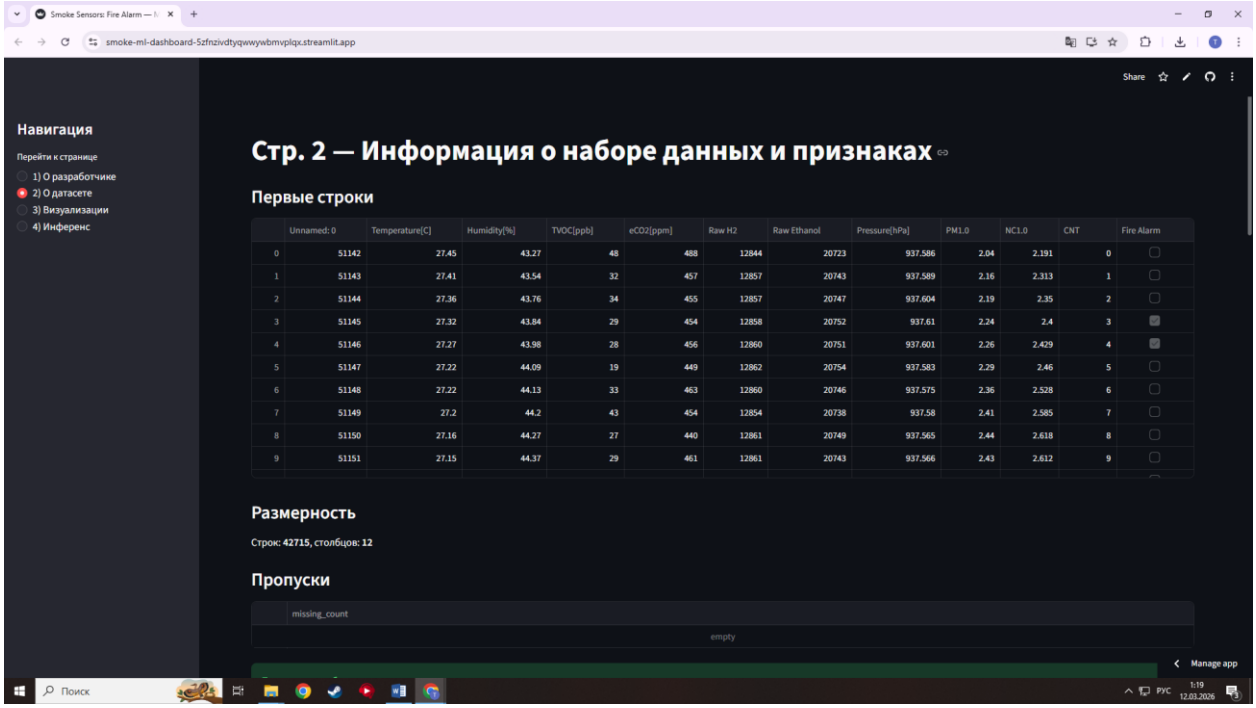



Рисунок А.2 – Страница описания датасета

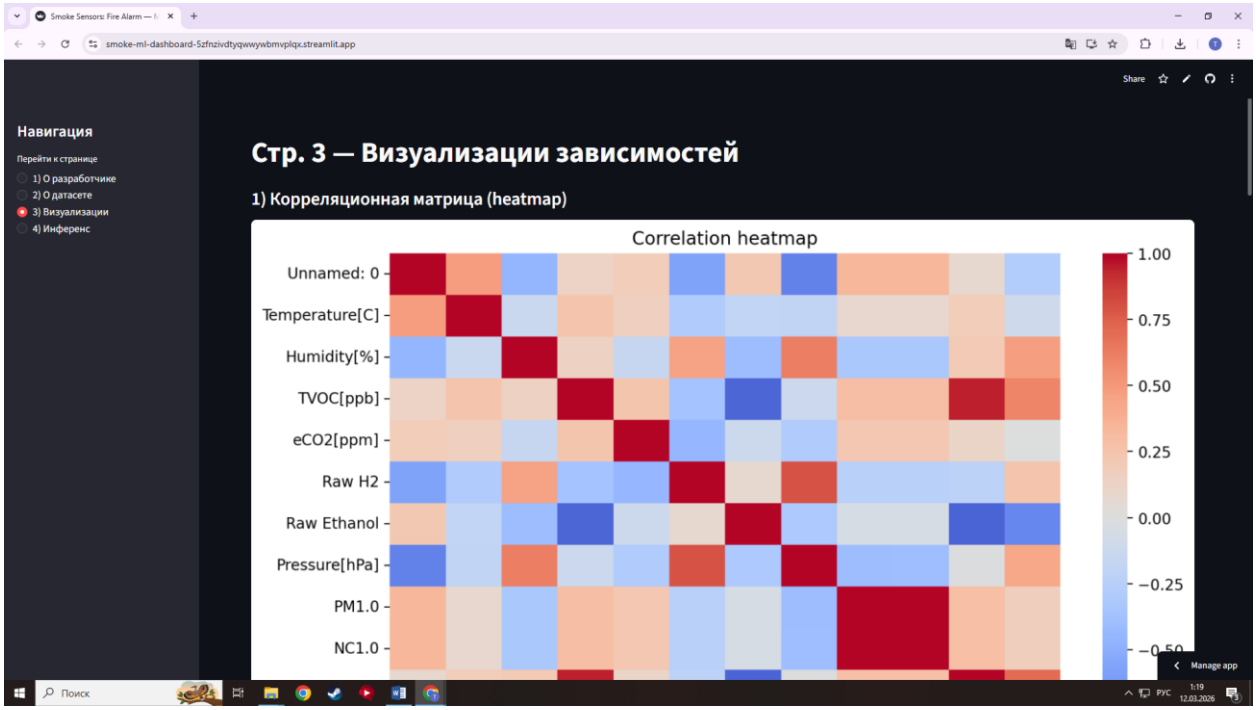



Рисунок А.3 – Страница визуального анализа данных

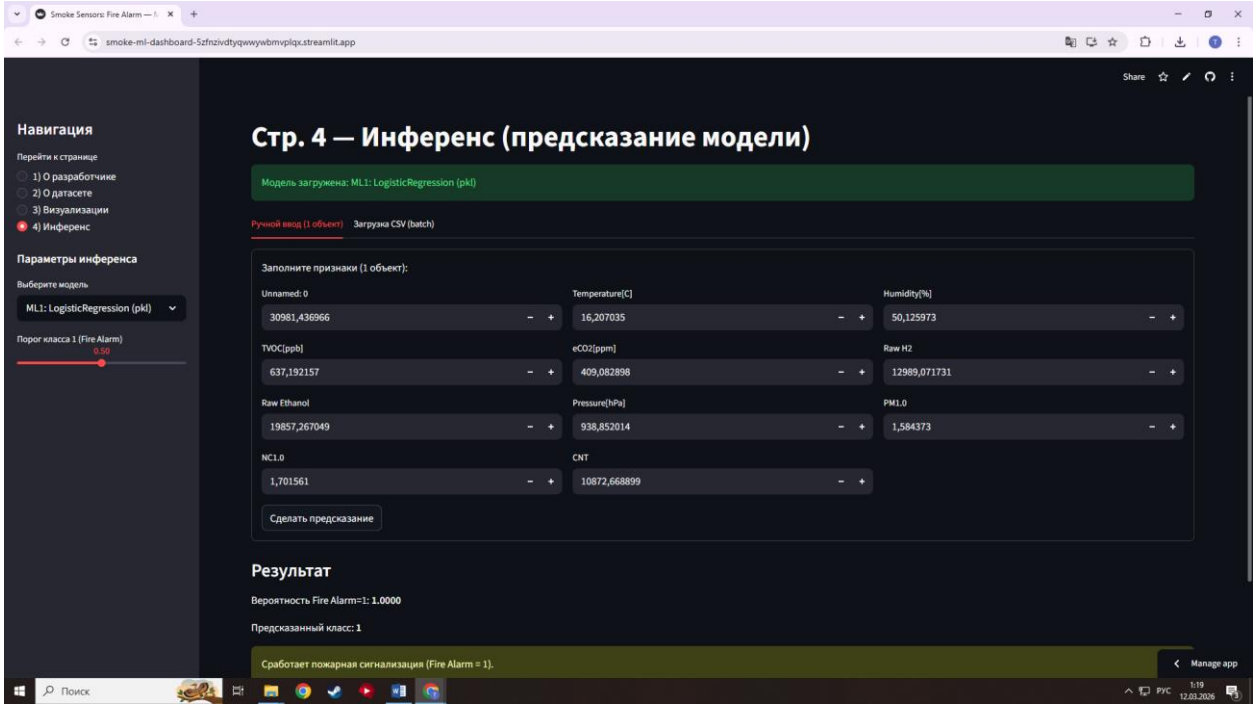



Рисунок А.4 – Страница инференса моделей и прогнозирования

# ПРИЛОЖЕНИЕ Б

(рекомендованное)

## Листинг кода

Файл app.py

```
import os
import numpy as np
import pandas as pd
import streamlit as st

import matplotlib.pyplot as plt
import seaborn as sns

import joblib

from catboost import CatBoostClassifier

# Настройки приложения
st.set_page_config(
    page_title="Smoke Sensors: Fire Alarm — ML Dashboard",
    layout="wide"
)

DATA_PATH = "smokeEDA.csv"
TARGET_COL = "Fire Alarm"

MODELS_DIR = "models"
ASSETS_DIR = ""

MODEL_PATHS = {
    "ML1:      LogisticRegression      (pkl)":      os.path.join(MODELS_DIR,
"ML1_LogisticRegression.pkl"),
    "ML2:       GradientBoosting       (pkl)":       os.path.join(MODELS_DIR,
"ML2_GradientBoostingClassifier.pkl"),
    "ML3:          CatBoost          (cbm)":          os.path.join(MODELS_DIR,
"ML3_CatBoostClassifier.cbm"),
    "ML4:           Bagging          (pkl)":          os.path.join(MODELS_DIR,
"ML4_BaggingClassifier.pkl"),
    "ML5:          Stacking          (pkl)":          os.path.join(MODELS_DIR,
"ML5_StackingClassifier.pkl"),

}


# Загрузка данных
@st.cache_data
def load_data(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
```

```
    return df


def get_feature_columns(df: pd.DataFrame) -> list[str]:
    if TARGET_COL not in df.columns:
        raise ValueError(f"Целевая колонка '{TARGET_COL}' не найдена. Колонки: {list(df.columns)}")
    feature_cols = [c for c in df.columns if c != TARGET_COL]
    return feature_cols


# Загрузка моделей
@st.cache_resource
def load_sklearn_model(path: str):
    return joblib.load(path)


@st.cache_resource
def load_catboost_model(path: str) -> CatBoostClassifier:
    model = CatBoostClassifier()
    model.load_model(path)
    return model



def is_catboost(model_key: str) -> bool:
    return "CatBoost" in model_key


def is_tf_nn(model_key: str) -> bool:
    return "Dense NN" in model_key or "TensorFlow" in model_key


def get_model(model_key: str):
    path = MODEL_PATHS[model_key]
    if not os.path.exists(path):
        raise FileNotFoundError(f"Файл/папка модели не найден(а): {path}")

    if is_tf_nn(model_key):
        raise FileNotFoundError(
            "TF-модель (ML6) ещё не подключена. "
            "Добавьте файл/папку модели и раскомментируйте строки в коде."
        )

    if is_catboost(model_key):
        return load_catboost_model(path)

    return load_sklearn_model(path)


# Предсказание:
# Возвращает вероятность класса 1 (Fire Alarm = 1)
def predict_proba(model, X: pd.DataFrame, model_key: str) -> np.ndarray:
    if is_tf_nn(model_key):
        raise RuntimeError("TF-модель не подключена (см. комментарии в коде).")
```

```
    # sklearn / catboost
    if hasattr(model, "predict_proba"):
        return model.predict_proba(X)[:, 1]

    preds = model.predict(X)
    return np.asarray(preds).astype(float)


def predict_label_from_proba(proba: np.ndarray, threshold: float = 0.5) ->
np.ndarray:
    return (proba >= threshold).astype(int)


# UI: ввод одной строки
def build_single_row_input(df: pd.DataFrame, feature_cols: list[str]) ->
pd.DataFrame:
    desc = df[feature_cols].describe(include="all").T
    defaults = df[feature_cols].mean(numeric_only=True)

    with st.form("single_row_form"):
        st.write("Заполните признаки (1 объект):")
        cols = st.columns(3)
        values = {}

        for i, c in enumerate(feature_cols):
            col = cols[i % 3]

            if pd.api.types.is_numeric_dtype(df[c]):
                c_min = float(desc.loc[c, "min"]) if "min" in desc.columns
else float(df[c].min())
                c_max = float(desc.loc[c, "max"]) if "max" in desc.columns
else float(df[c].max())
                c_def = float(defaults.get(c, (c_min + c_max) / 2.0))

                step = (c_max - c_min) / 100.0 if c_max > c_min else 1.0
                step = float(step) if np.isfinite(step) and step > 0 else 1.0

                values[c] = col.number_input(
                    label=f"{c}",
                    value=c_def,
                    min_value=c_min,
                    max_value=c_max,
                    step=step,
                    format="%.6f"
                )
            else:
                values[c]              =              col.text_input(f"{c}",
value=str(df[c].mode().iloc[0]))

        submitted = st.form_submit_button("Сделать предсказание")
        if not submitted:
            return None

    return pd.DataFrame([values], columns=feature_cols)
```

```
# Страницы
def page_about():
    st.title("Стр. 1 — Информация о разработчике")

    left, right = st.columns([1, 2])

    with left:
        photo_path = os.path.join(ASSETS_DIR, "photo.jpg")
        if os.path.exists(photo_path):
            st.image(photo_path, caption="Фото", use_container_width=True)
        else:
            st.info("Добавьте фото в assets/photo.jpg (необязательно).")

    with right:
        st.markdown(
            """
**ФИО:** Смаилов Тимур
**Группа:** ФИТ-231
**Дисциплина:** Машинное обучение и большие данные
**Тема РГР:** Разработка Web-приложения (дашборда) для инференса моделей ML и
анализа данных
**Датасет:** smokeEDA.csv (датчики дыма)
**Цель:** классификация целевого признака `Fire Alarm` (Бинарный)
"""
        )


def page_dataset_info(df: pd.DataFrame):
    st.title("Стр. 2 — Информация о наборе данных и признаках")

    st.subheader("Первые строки")
    st.dataframe(df.head(20), use_container_width=True)

    st.subheader("Размерность")
    st.write(f"Строк: **{df.shape[0]}**, столбцов: **{df.shape[1]}**")

    st.subheader("Пропуски")
    na = df.isna().sum().sort_values(ascending=False)
    st.dataframe(na[na             >             0].to_frame("missing_count"),
use_container_width=True)
    if (na == 0).all():
        st.success("Пропусков не обнаружено.")

    st.subheader("Типы данных")
    st.dataframe(df.dtypes.to_frame("dtype"), use_container_width=True)

    if TARGET_COL in df.columns:
        st.subheader("Распределение целевого класса")
        vc = df[TARGET_COL].value_counts(dropna=False)
        st.write(vc)

        fig, ax = plt.subplots(figsize=(6, 4))
        sns.countplot(x=TARGET_COL, data=df, ax=ax)
        ax.set_title("Target distribution")
        st.pyplot(fig, clear_figure=True)
```

```
    st.subheader("Описательная статистика (числовые признаки)")
    st.dataframe(df.describe().T, use_container_width=True)



def page_visuals(df: pd.DataFrame):
    st.title("Стр. 3 — Визуализации зависимостей")

    feature_cols = get_feature_columns(df)

    # 1) Heatmap корреляций
    st.subheader("1) Корреляционная матрица (heatmap)")
    corr = df[feature_cols + [TARGET_COL]].corr(numeric_only=True)

    fig, ax = plt.subplots(figsize=(10, 6))
    sns.heatmap(corr, cmap="coolwarm", center=0, ax=ax)
    ax.set_title("Correlation heatmap")
    st.pyplot(fig, clear_figure=True)

    # 2) Boxplot по одному признаку относительно класса
    st.subheader("2) Boxplot признака по классам")
    box_feature = st.selectbox("Выберите признак для boxplot", feature_cols,
index=0)
    fig, ax = plt.subplots(figsize=(8, 4))
    sns.boxplot(data=df, x=TARGET_COL, y=box_feature, ax=ax)
    ax.set_title(f"{box_feature} by {TARGET_COL}")
    st.pyplot(fig, clear_figure=True)

    # 3) Scatterplot (2 признака) с hue=target
    st.subheader("3) Scatterplot (2 признака) с раскраской по классу")
    c1, c2 = st.columns(2)
    with c1:
        x_feat = st.selectbox("X", feature_cols, index=0, key="scatter_x")
    with c2:
        y_feat     =     st.selectbox("Y",     feature_cols,     index=min(1,
len(feature_cols) - 1), key="scatter_y")

    fig, ax = plt.subplots(figsize=(8, 5))
    sns.scatterplot(data=df,  x=x_feat,  y=y_feat,  hue=TARGET_COL,  alpha=0.6,
ax=ax)
    ax.set_title(f"{x_feat} vs {y_feat}")
    st.pyplot(fig, clear_figure=True)

    # 4) Гистограмма распределения выбранного признака
    st.subheader("4) Гистограмма распределения признака")
    hist_feature  =  st.selectbox("Признак  для  гистограммы",  feature_cols,
index=0, key="hist")
    fig, ax = plt.subplots(figsize=(8, 4))
    sns.histplot(data=df, x=hist_feature, hue=TARGET_COL, bins=30, kde=True,
ax=ax)
    ax.set_title(f"Distribution of {hist_feature}")
    st.pyplot(fig, clear_figure=True)


def page_inference(df: pd.DataFrame):
```

```
    st.title("Стр. 4 — Инференс (предсказание модели)")

    feature_cols = get_feature_columns(df)

    st.sidebar.subheader("Параметры инференса")
    model_key = st.sidebar.selectbox("Выберите модель",
list(MODEL_PATHS.keys()))
    threshold = st.sidebar.slider("Порог класса 1 (Fire Alarm)", 0.0, 1.0,
0.5, 0.01)

    try:
        model = get_model(model_key)
        st.success(f"Модель загружена: {model_key}")
    except Exception as e:
        st.error(str(e))
        st.stop()

    tab1, tab2 = st.tabs(["Ручной ввод (1 объект)", "Загрузка CSV (batch)"])

    with tab1:
        X_one = build_single_row_input(df, feature_cols)
        if X_one is not None:
            proba = predict_proba(model, X_one, model_key=model_key)
            pred = predict_label_from_proba(proba, threshold=threshold)

            st.subheader("Результат")
            st.write(f"Вероятность Fire Alarm=1: **{float(proba[0]):.4f}**")
            st.write(f"Предсказанный класс: **{int(pred[0])}**")

            if int(pred[0]) == 1:
                st.warning("Сработает пожарная сигнализация (Fire Alarm =
1).")
            else:
                st.info("Пожарная сигнализация не сработает (Fire Alarm =
0).")

    with tab2:
        st.write("Загрузите CSV с теми же признаками, что и в обучении (без
колонки целевого класса).")
        up = st.file_uploader("CSV файл", type=["csv"])

        if up is not None:
            batch = pd.read_csv(up)

            if TARGET_COL in batch.columns:
                batch = batch.drop(columns=[TARGET_COL])

            missing = [c for c in feature_cols if c not in batch.columns]
            extra = [c for c in batch.columns if c not in feature_cols]

            if missing:
                st.error(f"В загруженном файле не хватает колонок:
{missing}")
                st.stop()

            if extra:
```

```
                st.warning(f"Лишние колонки будут проигнорированы: {extra}")

            Xb = batch[feature_cols].copy()

            proba = predict_proba(model, Xb, model_key=model_key)
            pred = predict_label_from_proba(proba, threshold=threshold)

            out = batch.copy()
            out["proba_fire_alarm_1"] = proba
            out["pred_fire_alarm"] = pred

            st.subheader("Предсказания (первые строки)")
            st.dataframe(out.head(50), use_container_width=True)

            st.download_button(
                "Скачать результаты (CSV)",
                data=out.to_csv(index=False).encode("utf-8"),
                file_name="predictions.csv",
                mime="text/csv"
            )


def main():
    if not os.path.exists(DATA_PATH):
        st.error(f"Файл датасета не найден: {DATA_PATH}")
        st.stop()

    df = load_data(DATA_PATH)

    st.sidebar.title("Навигация")
    page = st.sidebar.radio(
        "Перейти к странице",
        ["1) О разработчике", "2) О датасете", "3) Визуализации", "4)
Инференс"],
        index=0
    )

    if page == "1) О разработчике":
        page_about()
    elif page == "2) О датасете":
        page_dataset_info(df)
    elif page == "3) Визуализации":
        page_visuals(df)
    elif page == "4) Инференс":
        page_inference(df)


main()
```